Semantic Operators: A Declarative Model for Rich, AI-based Data Processing

Liana Patel†, Siddharth Jha‡, Melissa Pan‡, Harshit Gupta†, Parth Asawa‡,
Carlos Guestrin†, Matei Zaharia‡
† Stanford University, ‡ UC Berkeley
Correspondence: lianapat@stanford.edu
LOTUS Repository and Tutorials

arXiv:2407.11418v3 [cs.DB] 1 Mar 2025

ABSTRACT

The semantic capabilities of large language models (LLMs) have the potential to enable rich analytics and reasoning over vast knowledge corpora. Unfortunately, existing systems either empirically optimize expensive LLM-powered operations with *no performance guarantees*, or serve a limited set of *row-wise* LLM operations, providing limited robustness, expressiveness and usability. We introduce *semantic operators*, the first formalism for declarative and general-purpose AI-based transformations based on natural language specifications (e.g., filtering, sorting, joining or aggregating records using natural language criteria). Each operator opens a rich space for execution plans, similar to relational operators. Our model specifies the expected behavior of each operator with a high-quality gold algorithm, and we develop an optimization framework that reduces cost, while providing accuracy guarantees with respect to a gold algorithm. Using this approach, we propose several novel optimizations to accelerate semantic filtering, joining, group-by and top-k operations by up to $1,000\times$. We implement semantic operators in the LOTUS system and demonstrate LOTUS' effectiveness on real, bulk-semantic processing applications, including fact-checking, biomedical multi-label classification, search, and topic analysis. We show that the semantic operator model is expressive, capturing state-of-the-art AI pipelines in a few operator calls, and making it easy to express new pipelines that match or exceed quality of recent LLM-based analytic systems by up to 170%, while offering accuracy guarantees. Overall, LOTUS programs match or exceed the accuracy of state-of-the-art AI pipelines for each task while running up to $3.6\times$ faster than the highest-quality baselines. LOTUS is publicly available at https://github.com/lotus-data/lotus.

1 INTRODUCTION

The powerful semantic capabilities of modern language models (LLMs) create exciting opportunities for building AI-based analytics systems that reason over vast knowledge corpora. Many applications require complex reasoning over large amounts of data, including both unstructured and structured data. For example a researcher reviewing recent ArXiv [2] preprints may want to quickly obtain a summary of relevant papers from the past week, or find the papers that report the best performance for a particular task and dataset. Similarly, a medical professional may automatically extract biomedical characteristics and candidate diagnoses from many patient reports [29]. Likewise, organizations wish to automatically digest lengthy transcripts from internal meeting transcripts and chat histories to validate hypotheses about their business [4].

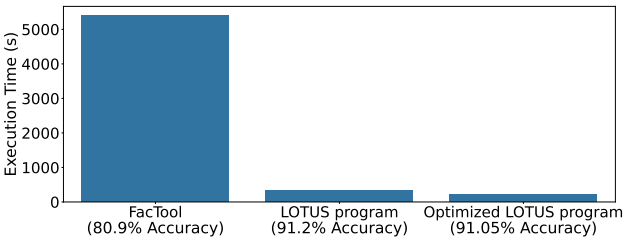

Figure 1: Execution time (y-axis) and accuracy, shown in parentheses, for 3 fact-checking implementations: (i) FacTool [23], a recent state-of-the-art research work, (ii) a short, un-optimized LOTUS program, and (iii) the same LOTUS program optimized with accuracy guarantees. Section 5 provides our full methodology.

Each of these tasks require a form of *bulk semantic processing*, where the analytics system must process large amounts of data and apply semantic-based analysis across a whole dataset. Supporting the full generality of these applications with efficient and easy-to-use analytics systems would have a transformative impact, similar to what RDBMSes had for tabular data. This prospect, however, raises two challenging questions: first, *how should developers express semantic queries*, and secondly, *how should we design the underlying analytics system to achieve high efficiency and accuracy*.

Unfortunately, existing systems are insufficient for bulk semantic processing, either limiting their expressiveness to row-wise LLM execution models or focusing on empirical optimizations without any formalism or accuracy guarantees, leading to unexpected behaviors and frequent failures. First, several systems provide logically row-wise LLM operators, either in the form of a general-purpose AI user-defined functions (UDF) [1, 8, 9, 14, 49, 53, 67] or specialized row-wise operators [46, 47, 50] for specific applications, such as conversational chat, data cleaning and extract-transform-load (ETL) tasks. These programming models fail to support a range of LLM-based transformations *across* rows, such as ranking, grouping or joining records. Alternatively, more recent LLM-based analytics systems, such as DocETL [61] and UQE [25], empirically optimize LLM-based operations with no accuracy guarantees, lacking any formalism to define correct behavior, which hinders their robustness and usability, as we show in Section 5. These obstacles highlight a core challenge of integrating semantic-based processing within reliable query systems due to the inherent ambiguity of natural language instructions and LLM outputs.

Towards robust, declarative query systems for bulk semantic processing, we propose the *semantic operator model*, which extends

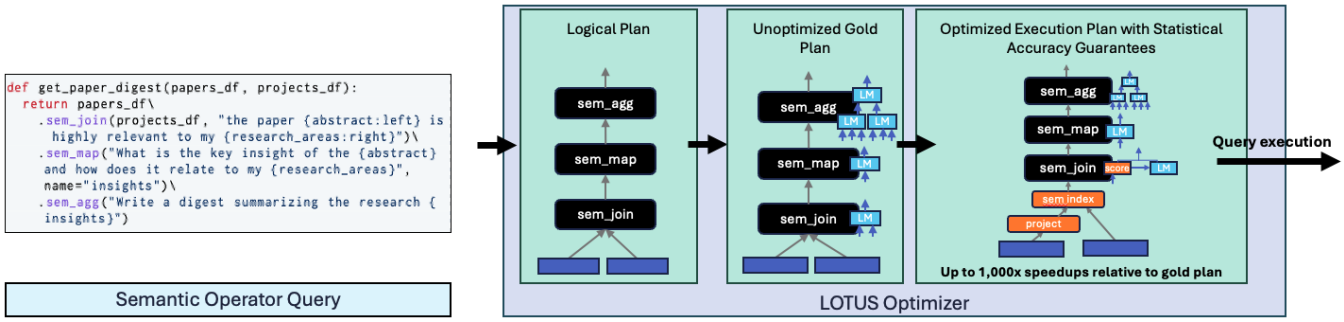

Figure 2: An example semantic operator query is shown on the left. The user function takes two two datasets and uses a series of semantic operators to to create a summary of relevant research papers from papers_df based on the user's research areas from projects_df. Given a user's query, LOTUS defines a logical plan, which it then converts to an un-optimized gold plan, that specifies how to orchestrate the LM over the data. LOTUS then generates an optimized execution plan with statistical accuracy guarantees relative to the gold plan. These optimized plans empirically demonstrate up to $1,000\times$ speedups. Finally, LOTUS executes the optimized plan and returns the results to the user.

the relational model with AI-based operations. Semantic operators provide the first formalism for general-purpose and multi-row, AI-based transformation, including semantic filters, joins, top-k rankings, aggregations, and projections. Each operator takes a concise natural language signature, given by the programmer, and its behavior is fully specified by a tractable, high-quality "gold" algorithm, which indicates how to orchestrate the underlying AI model over the data. Our optimization approach then exploits the rich design space of semantic operator execution plans to reduce cost, while providing *statistical accuracy guarantees* with respect to the gold algorithm (i.e., guarantees that the output of the optimized operator will be similar to that of the gold algorithm). We propose several novel optimizations for the semantic filter, join, top-k and group-by operators. Our methods include approximation algorithms, building on statistical techniques used in prior works [37, 39] with novel and efficient proxy scorers using small LLMs or semantic embeddings. We implement semantic operators in LOTUS (**L**LMs **O**ver **T**ables of **U**nstructured and **S**tructured data), an open source system that exposes these operators through a simple DataFrame-based programming interface.

Figure 1 illustrates the power of LOTUS' declarative programming model and optimized query engine. For a fact-checking task on the FEVER dataset [64], we write an intuitive LOTUS program to re-implement a recent state-of-the-art fact-checking pipeline, FacTool [23], which was originally implemented in over 750 lines of code. Our un-optimized LOTUS program composes 3 semantic operators (filters, maps and joins) and achieves 12.7% higher accuracy on this task. LOTUS also provides query efficiency by default, yielding $28\times$ lower execution time than the original FacTool implementation. Furthermore, the optimized LOTUS program, configured with 0.9 accuracy targets and a 0.2 allowed error probability, runs $1.7\times$ faster and obtains 99.8% accuracy compared to the un-optimized LOTUS program.

We systematically evaluate LOTUS on four real bulk-semantic processing applications: fact-checking, biomedical multi-label classification, search, and topic analysis. These include applications expressible and studied by recent LLM-based analytics systems [25, 61], where we compare against those systems, as well as others, where we compare against hand-designed pipelines from the AI literature. We demonstrate that the semantic operator model is effective, capturing state-of-the-art AI pipelines in a few operator calls that reliably match or exceed quality of recent LLM-based analytics systems by up to 100%, while offering accuracy guarantees and running up to $3.6\times$ faster than the best-quality baselines. Furthermore, our gold algorithms provide high-quality results, and our optimizations speed up query execution by up to $1,000\times$ with respect to them. Specifically, our LOTUS program re-implements FacTool's recent state-of-the-art fact-checking pipeline [23], as shown in Figure 1, and achieves 12.5% higher accuracy with $28\times$ lower execution time on the FEVER dataset [64], outperforming alternative LLM-powered analytics systems on both accuracy and efficiency. In the extreme multi-label classification task on the BioDEX dataset [29], LOTUS reproduces state-of-the art result quality [28] and the LLM-powered analytics baselines, while providing accuracy guarantees and $1,000\times$ speedups with respect to our gold algorithm. In the search and ranking application, LOTUS allows a simple composition of operators to achieve $3-100\%$ higher nDCG@10 than the next best performing baselines, while also providing query efficiency, with $1.67-10\times$ lower execution time than alternative high-quality algorithms. In the topic analysis task, LOTUS efficiently processes hundreds of ArXiv papers to discover a taxonomy of key topics in dozens of seconds, while once again providing configurable speedups with statistical accuracy guarantees.

Overall, our main contributions are the following:

- We propose semantic operators, the first formalized programming model for general-purpose AI-based transformations with natural language specifications.
- We define an optimization framework for semantic operators with statistical accuracy guarantees, and novel optimizations for several operators, including semantic filters, joins, top-k and group-by, yielding up to $1,000\times$ speedups.
- We extensively evaluate the expressiveness of the semantic operator model, the quality of our gold algorithms, and the benefit of our optimizations, including comparisons to DocETL, UQE, and AI UDFs.

Table 1: Summary of Key Semantic Operators. T **denotes a relation,** X **and** Y **denote arbitrary tuple types.** l **denotes a parameterized natural language expression ("langex" for short). Each operator may permit additional optional parameters, including accuracy targets.**

Operator	Description	Definition	Gold Algorithm
$sem_filter(l{:}\, X \rightarrow Bool)$	Returns the tuples that pass the langex predicate.	$\{t_i \in T \mid l_M(t_i) = 1\}$	Compute $M(t_i, l) \forall t_i \in T$
$sem_join(t{:}\, T,\ \ l{:}\, (X, Y) \rightarrow Bool)$	Joins a table against a second table t by keeping all tuple pairs that pass the langex predicate.	$\{(t_i, t_j) \mid l_M(t_i, t_j) = 1, t_i \in T_1, t_j \in T_2\}$	Compute $M(t_i, t_j, l) \forall t_i \in T_1, t_j \in T_2$
$sem_agg(l{:}\, T[X] \rightarrow X)$	Aggregates input tuples according to the langex reducer function.	$l_M(t_1, ..., t_n) \forall t_1, .., t_n \in T$	Perform a reduce algorithm, recursively computing $a_{i+1,j} = M(a_{i,f(j)}, ..., a_{i,f(j)+n'}, l)$, $a_{0,j} = M(t_{f(j)}, ..., t_{f(j)+n'}, l)$
$sem_topk(l{:}\, T[X] \rightarrow Seq[X],\ \ k{:}\, int)$	Returns an ordered list of the k best tuples according to the langex ranking criteria.	$\langle t_1, ..., t_k \rangle\ st\ \forall (t_i, t_j), i < j \implies l_M(t_i, t_j) = \langle t_i, t_j \rangle$	Perform top-k sorting algorithm using pairwise comparisons, $M(t_i, t_j, l)$
$sem_group_by(l{:}\, X \rightarrow Y,\ \ C{:}\, int)$	Groups the tuples into C categories based on the langex grouping criteria.	$\underset{\{\mu_1, ...\mu_C\}, \mu_i \in V^{\mathbb{N}}}{\arg\max} \sum_{t_i \in T} \max_{j \in 1...C} l_M(t_i, \mu_j)$	Obtain centers $\mu_1, ...\mu_C$ with a clustering algorithm, and perform pointwise assignments $M(t_i, \mu_1, ..., \mu_C) \forall t_i \in T$
$sem_map(l{:}\, X \rightarrow Y)$	Performs the projection specified by the langex.	$\{l_M(t_i), \forall t_i \in T\}$	Compute $M(t_i, l) \forall t_i \in T$

2 THE SEMANTIC OPERATOR MODEL

Semantic operators provide a declarative interface for semantic-based manipulation and access to data. Extending the data independence model of relational systems [24], these new *AI-based* operators introduce *model-data independence*, which separates the application logic from the underlying ML-based algorithm and access pattern that specifies how data is passed to model invocations. Unlike relational operators, semantic operators are parametrized by natural language expressions and rely on AI-based computation, making their behavior inherently ambiguous and imprecise. We address this key challenge by presenting a formalism for defining semantic operators, their behavior and correct optimizations, in this section. In Section 3, we build on this formalism to provide optimized execution plans with accuracy guarantees, similar to relational query optimizers.

2.1 Defining Semantic Operators

Definition. *A semantic operator is a declarative transformation over one or more datasets, parameterized by a natural language expression. Each semantic operator can be implemented by many AI-based algorithms, and its behavior is defined with respect to a gold algorithm.*

Table 1 lists a core set of semantic operators, which cover common semantic transformations in real-world applications and mirror key transformations in relational operators. Specific systems may, of course, provide additional semantic operators beyond the ones we discuss here. Each semantic operator takes a *parameterized natural language expressions* (*langex* for short), which are natural language expressions that specify a function over one or more attributes. As Figure 2 demonstrates, the langex signature varies for different semantic transformations. The figure shows a LOTUS program, which performs a `sem_join`, `sem_map` and `sem_agg` over datasets of research papers and user projects. While the `sem_join` langex signature provides a natural language *predicate*, the `sem_map` langex signature indicates a natural language projection, and the `sem_agg` langex is a commutative, associative *aggregator* expression, which here indicates a summarization task over abstracts.

We provide a high-level definition of each semantic transformation with respect to the user langex and a world model[1], M, which captures a probability distribution over the vocabulary V. For example, semantic filter returns $\{t_i \in T \mid l_M(t_i) = 1\}$, where T is the input relation and $l_M(t_i)$ represents a natural language predicate evaluated on tuple t_i with model M. Notably, the definition of each semantic operator can be implemented by multiple AI-based algorithms, and different decisions as to how to invoke the model over the relation have consequences on the algorithm's result quality. Thus, to fully specify the ground truth behavior of each semantic operator, we define a *gold algorithm*, which is a computable and tractable ML algorithm that produces high-quality results and specifies the model's access pattern over the data, $M(\cdot)$.

2.2 Defining Correct Optimizations for Semantic Operators

Semantic operators create a rich design space of diverse execution plans. While gold algorithms define a tractable, high-quality implementation, they are often expensive, with the complexity of LLM calls scaling linearly or quadratically in the dataset cardinality. For example, the semantic filter's gold algorithm entails invoking the LLM in a linear pass over each tuple. However, an alternate AI-based algorithm may reduce cost, while offering *close results*. In the case of semantic filters, *close results*, can be measured according to accuracy metrics, in this case recall and precision, and guaranteed with high probability compared to the accuracy obtained by the gold algorithm. As such, we define correct optimizations for semantic operators[2] below.

Definition. *A correct optimization for a given semantic operator, and a gold algorithm for that operator, reduces cost while providing statistical accuracy guarantees with respect to the gold algorithm. Specifically, the optimization should ensure an accuracy target,* γ*, is met with probability* $1 - \delta$*.*

In general, optimized semantic operator plans allows for both lossless optimizations and approximations with respect to a gold algorithm. This formulation builds on approaches in approximate query processing and assumes some error is often tolerable, which

[1] In practice, the world model, M may be the strongest LLM a practioner has available.

[2] Our optimization approach can naturally extend to multi-operator execution plans by composition. In multi-operator programs, the user may configure an accuracy target and failure probability per operator, or for the full program, in which case the optimizer assigns a target accuracy and failure budget to each operator.